

3013. Divide an Array Into Subarrays With Minimum Cost II

Solved 

Hard

Topics

Companies

Hint

You are given a **0-indexed** array of integers `nums` of length `n`, and two **positive** integers `k` and `dist`.

The **cost** of an array is the value of its **first** element. For example, the cost of `[1,2,3]` is `1` while the cost of `[3,4,1]` is `3`.

You need to divide `nums` into `k` **disjoint contiguous subarrays**, such that the difference between the starting index of the **second** subarray and the starting index of the `k`th subarray should be **less than or equal to** `dist`. In other words, if you divide `nums` into the subarrays `nums[0..(i1 - 1)]`, `nums[i1..(i2 - 1)]`, ..., `nums[ik-1..(n - 1)]`, then $i_{k-1} - i_1 \leq \text{dist}$.

Return the **minimum** possible sum of the cost of these subarrays.

Example 1:

Input: `nums = [1,3,2,6,4,2]`, `k = 3`, `dist = 3`

Output: `5`

Explanation: The best possible way to divide `nums` into 3 subarrays is: `[1,3]`, `[2,6,4]`, and `[2]`. This choice is valid because $i_{k-1} - i_1$ is $5 - 2 = 3$ which is equal to `dist`. The total cost is `nums[0] + nums[2] + nums[5]` which is $1 + 2 + 2 = 5$. It can be shown that there is no possible way to divide `nums` into 3 subarrays at a cost lower than 5.

Example 2:

Input: `nums = [10,1,2,2,2,1]`, `k = 4`, `dist = 3`

Output: `15`

Explanation: The best possible way to divide `nums` into 4 subarrays is: `[10]`, `[1]`, `[2]`, and `[2,2,1]`. This choice is valid because $i_{k-1} - i_1$ is $3 - 1 = 2$ which is less than `dist`. The total cost is `nums[0] + nums[1] + nums[2] + nums[3]` which is $10 + 1 + 2 + 2 = 15$.

The division `[10]`, `[1]`, `[2,2,2]`, and `[1]` is not valid, because the difference between i_{k-1} and i_1 is $5 - 1 = 4$, which is greater than `dist`.

It can be shown that there is no possible way to divide `nums` into 4 subarrays at a cost lower than 15.

Example 3:

Input: `nums = [10,8,18,9]`, `k = 3`, `dist = 1`

Output: `36`

Explanation: The best possible way to divide `nums` into 3 subarrays is: `[10]`, `[8]`, and `[18,9]`. This choice is valid because $i_{k-1} - i_1$ is $2 - 1 = 1$ which is equal to `dist`. The total cost is `nums[0] + nums[1] + nums[2]` which is $10 + 8 + 18 = 36$.

The division `[10]`, `[8,18]`, and `[9]` is not valid, because the difference between i_{k-1} and i_1 is $3 - 1 = 2$, which is greater than `dist`.

It can be shown that there is no possible way to divide `nums` into 3 subarrays at a cost lower than 36.

Constraints:

- $3 \leq n \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^9$
- $3 \leq k \leq n$
- $k - 2 \leq \text{dist} \leq n - 2$

Python:

class Solution:

class SmartWindow:

def __init__(self, k: int):

self.K = k

self.low = [] # sorted list

self.high = [] # sorted list

self.sumLow = 0

def windowSize(self) -> int:

return len(self.low) + len(self.high)

def _erase_one(self, arr: List[int], x: int) -> bool:

i = bisect_left(arr, x)

if i < len(arr) and arr[i] == x:

arr.pop(i)

return True

return False

def rebalance(self) -> None:

need = min(self.K, self.windowSize())

while len(self.low) > need:

x = self.low.pop() # largest in low

self.sumLow -= x

insort(self.high, x)

while len(self.low) < need and self.high:

x = self.high.pop(0) # smallest in high

insort(self.low, x)

self.sumLow += x

```

def add(self, x: int) -> None:
    if not self.low or x <= self.low[-1]:
        insert(self.low, x)
        self.sumLow += x
    else:
        insert(self.high, x)
    self.rebalance()

```

```

def remove(self, x: int) -> None:
    if self._erase_one(self.low, x):
        self.sumLow -= x
    else:
        self._erase_one(self.high, x)
    self.rebalance()

```

```

def query(self) -> int:
    return self.sumLow

```

```

def minimumCost(self, nums: List[int], k: int, dist: int) -> int:
    n = len(nums)
    k -= 1
    window = self.SmartWindow(k)

```

```

    for i in range(1, 1 + dist + 1):
        window.add(nums[i])

```

```

    ans = window.query()

```

```

    for i in range(2, n - dist):
        window.remove(nums[i - 1])
        window.add(nums[i + dist])
        ans = min(ans, window.query())

```

```

    return ans + nums[0]

```

JavaScript:

```

/**
 * @param {number[]} nums
 * @param {number} k
 * @param {number} dist
 * @return {number}
 */
var minimumCost = function(nums, k, dist) {
    const n = nums.length;

```

```

const INF = Number.MAX_SAFE_INTEGER;

let sum = 0;
let ans = INF;
const used = new Set();

// max heap by value
const heapUsed = new MaxPriorityQueue({
  compare: (a, b) => b[0] - a[0]
});

// min heap by value
const heapUnused = new MinPriorityQueue({
  compare: (a, b) => a[0] - b[0]
});

for (let right = 1; right < n; right++) {
  const left = right - dist - 1;

  // move left border
  if (left > 0 && used.has(left)) {
    used.delete(left);
    sum -= nums[left];

    while (!heapUnused.isEmpty() && heapUnused.front()[1] < left) {
      heapUnused.dequeue();
    }

    if (!heapUnused.isEmpty()) {
      const [val, idx] = heapUnused.dequeue();
      heapUsed.enqueue([val, idx]);
      used.add(idx);
      sum += val;
    }
  }

  // move right border
  if (used.size < k - 1) {
    heapUsed.enqueue([nums[right], right]);
    used.add(right);
    sum += nums[right];
  } else {
    while (!heapUsed.isEmpty() && !used.has(heapUsed.front()[1])) {
      heapUsed.dequeue();
    }
  }
}

```

```

    }

    if (!heapUsed.isEmpty() && nums[right] < heapUsed.front()[0]) {
        const [val, idx] = heapUsed.dequeue();

        used.delete(idx);
        heapUnused.enqueue([val, idx]);

        heapUsed.enqueue([nums[right], right]);
        used.add(right);

        sum += nums[right] - val;
    } else {
        heapUnused.enqueue([nums[right], right]);
    }
}

if (left >= 0) {
    ans = Math.min(ans, sum);
}

return nums[0] + ans;
};

```

Java:

```

class Solution {
    public long minimumCost(int[] nums, int k, int dist) {

        long result = Long.MAX_VALUE, windowSum = 0;
        int n = nums.length;
        TreeSet<Integer> using = new TreeSet<>((a, b) -> nums[a] == nums[b] ? a - b : nums[a] -
nums[b]);
        TreeSet<Integer> waiting = new TreeSet<>((a, b) -> nums[a] == nums[b] ? a - b : nums[a] -
nums[b]);

        // Add dist + 1 num into the using set
        for(int i = 1; i <= dist + 1; i++){
            using.add(i);
            windowSum += nums[i];
        }
        // We need only k - 1 nums, move others to waiting set because we might need them in the
future
        while(using.size() > k - 1){

```

```

    int i = using.pollLast();
    windowSum -= nums[i];
    waiting.add(i);
}
// windowSum is the sum of k - 1 nums
result = Math.min(result, windowSum);
for(int i = 1; i + dist + 1 < n; i++){

    // window slided so add new num to the waiting set
    waiting.add(i + dist + 1);

    // i is the left most num in the window, will be removed from window

    if(using.contains(i)){
        // i is one of the k - 1 num
        // remove and update windowSum
        // poll one num from waiting set and add
        windowSum -= nums[i];
        using.remove(i);
        int j = waiting.pollFirst();
        windowSum += nums[j];
        using.add(j);
    }else{
        // i is not one of the k - 1 num, it is in the waiting set
        // remove from waiting
        // check minimum num of the waiting set is lower than maximum num of the using set
        // If so, add to the using set, remove from waiting set
        // update window accordingly
        waiting.remove(i);
        int j1 = waiting.first(), j2 = using.last();
        if(nums[j1] < nums[j2]){
            windowSum -= nums[j2];
            using.remove(j2);
            waiting.add(j2);

            windowSum += nums[j1];
            using.add(j1);
            waiting.remove(j1);
        }
    }
}

result = Math.min(result, windowSum);
}
return result + nums[0];}}

```